Project Report: Placement Drive Merit Ranking System

Chapter 1: Introduction

Campus placement drives are crucial events for students seeking employment opportunities upon graduation. These drives involve numerous companies assessing a large pool of candidates through various tests and interviews. Managing the results, especially for aptitude tests, can be a labor-intensive and time-consuming process for college administration. Manually processing results, applying cutoff criteria, and ranking students accurately is prone to human error and delays. Automating this process offers significant benefits, including increased efficiency, reduced errors, and faster generation of merit lists. This project aims to develop a system that automates the processing of placement drive aptitude test results, ensuring a fair and efficient ranking of students based on their performance.

Chapter 2: Python Concepts Used

This project leverages several fundamental Python concepts to achieve its objectives:

Type Casting (String to Int)

Type casting, or type conversion, is the process of changing a variable's data type from one to another. In this system, student scores are initially read from a text file as strings. To perform numerical comparisons and sorting operations, these string representations of scores must be converted into integers. Python's `int()` function facilitates this conversion, allowing for mathematical manipulation of the scores.

2D Lists

A 2D list, also known as a list of lists or a nested list, is a data structure that organizes data in a tabular format, similar to a spreadsheet or a matrix. Each element of the

main list is itself another list. In this project, a 2D list is used to store student data, where each inner list contains a student's name (string) and their corresponding score (integer). This structure allows for efficient storage and manipulation of multiple student records.

Selection Sort

Selection Sort is a simple, in-place comparison sorting algorithm. It works by repeatedly finding the minimum (or maximum) element from the unsorted part of the list and putting it at the beginning (or end) of the sorted part. The algorithm maintains two sub-arrays in a given array: one which is already sorted and another which is remaining unsorted. In this project, a modified Selection Sort is used to rank students. The primary sorting criterion is the score in descending order. In cases where two students have the same score, their names are used as a secondary sorting criterion in ascending alphabetical order to ensure a consistent and fair tie-breaking mechanism.

Pseudo-code for Selection Sort (Descending Order with Tie-breaking):

```
function selection_sort(data):
  n = length of data
  for i from 0 to n-1:
    max_idx = i
    for j from i+1 to n-1:
      if data[j].score > data[max_idx].score:
        max_idx = j
      else if data[j].score == data[max_idx].score:
        if data[j].name < data[max_idx].name:
          max_idx = j
    swap data[i] and data[max_idx]
  return data
```

Chapter 3: System Design

The Placement Drive Merit Ranking System follows a clear pipeline to process raw aptitude test results and generate a sorted merit list. The design ensures modularity and ease of understanding for each processing step.

Pipeline:

1. **Raw File Input:** The system begins by reading student aptitude test results from a plain text file (e.g., `aptitude_results.txt`). Each line in this file is expected to contain a student's name and score, separated by a comma.

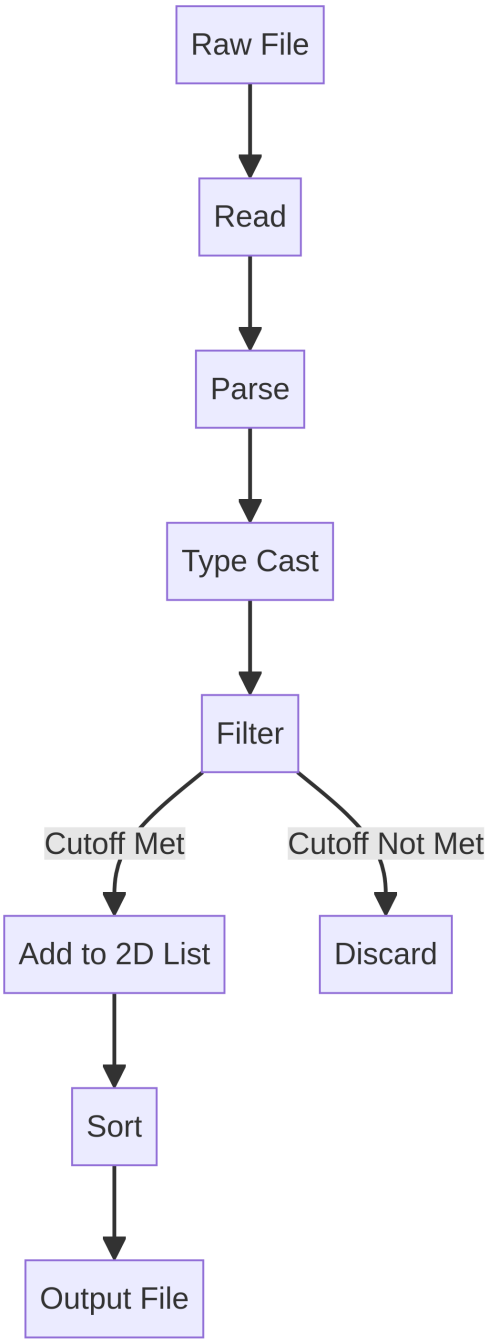
2. **Type Casting:** As data is read from the raw file, the scores, which are initially strings, are converted into integers using type casting. This step is crucial for enabling numerical comparisons and calculations.

3. **Cutoff Filter:** After type casting, each student's score is evaluated against a predefined cutoff mark. Only students who meet or exceed this cutoff score are considered for the merit list, effectively filtering out ineligible candidates.

4. **2D List Storage:** The filtered student data (name and score) is then stored in a 2D list. This data structure provides an organized way to manage multiple student records before sorting.

5. **Selection Sort:** The 2D list containing eligible students is then passed to the Selection Sort algorithm. This algorithm sorts the students primarily by score in descending order and secondarily by name in ascending order for tie-breaking.

6. **Formatted Output File:** Finally, the sorted data is written to a new output file (e.g., `merit_list.txt`) in a well-structured and readable format, including rank, name, and score for each student.



Chapter 4: Implementation

The Python code below implements the described system design. It includes functions for the Selection Sort algorithm and the overall result processing pipeline.

```python
def selection_sort(data):
    """
    Sorts a 2D list based on the score (index 1) in descending order.
    If scores are equal, sorts by name (index 0) in ascending order.
    """
    n = len(data)
    # Nested for loop logic for Selection Sort
    for i in range(n):
        max_idx = i
        for j in range(i + 1, n):
            # Compare scores (descending)
            if data[j][1] > data[max_idx][1]:
                max_idx = j
            # If scores are equal, compare names (ascending)
            elif data[j][1] == data[max_idx][1]:
                if data[j][0] < data[max_idx][0]:
                    max_idx = j

        # Swap the found maximum element with the first element
        data[i], data[max_idx] = data[max_idx], data[i]
    return data

def process_placement_results(input_file, output_file, cutoff=50):
    try:
        raw_data = []
        # Step 1: Read Raw File
        with open(input_file, 'r') as f:
            for line in f:
                parts = line.strip().split(',')
                if len(parts) == 2:
                    name = parts[0].strip()
                    # Step 2: Type Casting (String to Int)
                    score = int(parts[1].strip())

                    # Step 3: Cutoff Filter
                    if score >= cutoff:
                        raw_data.append([name, score]) # Step 4: 2D List

        # Step 5: Selection Sort
        sorted_data = selection_sort(raw_data)

        # Step 6: Formatted Output File
        with open(output_file, 'w') as f:
            f.write(f"{'Rank':<5} | {'Name':<20} | {'Score':<5}\n")
            f.write("-" * 35 + "\n")
```

```
            for rank, (name, score) in enumerate(sorted_data, 1):
                f.write(f"{rank:<5} | {name:<20} | {score:<5}\n")

        print(f"Successfully processed {len(sorted_data)} students.")
    except FileNotFoundError:
        print("Error: Input file not found.")
    except ValueError as e:
        print(f"Error in data format: {e}")

if __name__ == "__main__":
    # Sample execution
    process_placement_results('aptitude_results.txt', 'merit_list.txt')
```

Nested for loop logic in `selection_sort` function:

The core of the Selection Sort algorithm lies within its nested `for` loops. The outer loop iterates from `i = 0` to `n-1`, where `n` is the total number of elements. In each iteration of the outer loop, `max_idx` is initialized to `i`, assuming the current element is the maximum. The inner loop, iterating from `j = i + 1` to `n-1`, then compares `data[j]` with `data[max_idx]`. If a larger score is found (or an alphabetically smaller name for tie-breaking), `max_idx` is updated. After the inner loop completes, `max_idx` holds the index of the true maximum element in the unsorted portion. This maximum element is then swapped with the element at the current `i` position, effectively placing the largest element in its correct sorted position. This process continues until the entire list is sorted.

Chapter 5: Results

`aptitude_results.txt`

This file contains the raw aptitude test scores for various students before processing.

```
Alice, 85
Bob, 45
Charlie, 92
David, 78
Eve, 92
Frank, 60
Grace, 30
Hannah, 85
Ivan, 55
Jack, 70
```

`merit_list.txt`

This file presents the processed and sorted merit list, demonstrating the system's output after applying the cutoff filter and Selection Sort algorithm.

```
Rank   | Name                  | Score
-----------------------------------
1      | Charlie               | 92
2      | Eve                   | 92
3      | Alice                 | 85
4      | Hannah                | 85
5      | David                 | 78
6      | Jack                  | 70
7      | Frank                 | 60
8      | Ivan                  | 55
```

Chapter 6: Conclusion

This project successfully implemented a Placement Drive Merit Ranking System using Python, demonstrating the practical application of algorithmic sorting on real-world file data. The system effectively automates the process of reading raw aptitude test scores, applying a cutoff filter, and generating a ranked merit list. By utilizing concepts such as type casting, 2D lists, and the Selection Sort algorithm with custom tie-breaking logic, the solution provides an efficient, accurate, and robust method for managing placement drive results. This automation significantly reduces the administrative burden on educational institutions, minimizes errors, and ensures a

transparent and fair ranking process for students participating in campus placement drives. The project highlights the power of simple yet effective algorithms in solving practical data processing challenges.